

FRED TALKS

23rd August 2026

CONTENTS

Highlights from my conversation about agentic engineering on
Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 3349 WORDS

How not to forget what matters

HENRIK KARLSSON · 2660 WORDS

2.1.241

CHANGELOG · 5 WORDS

2.1.240

CHANGELOG · 5 WORDS

Highlights from my conversation about agentic engineering on Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 02 APR 2026 · [SOURCE](#)

Highlights from my conversation about agentic engineering on Lenny's Podcast

I was a guest on Lenny Rachitsky's podcast, in a new episode titled [An AI state of the union: We've passed the inflection point, dark factories are coming, and automation timelines](#). It's available on [YouTube](#), [Spotify](#), and [Apple Podcasts](#). Here are my highlights from our conversation, with relevant links.

[An AI state of the union: We've passed the inflection point & dark factories are coming](#)[Lenny's Podcast](#)



Lenny's Podcast 571K subscribers

[Watch on](#)

- [The November inflection point](#)
- [Software engineers as bellwethers for other information workers](#)
- [Writing code on my phone](#)
- [Responsible vibe coding](#)
- [Dark Factories and StrongDM](#)
- [The bottleneck has moved to testing](#)
- [This stuff is exhausting](#)
- [Interruptions cost a lot less now](#)

- My ability to estimate software is broken
- It's tough for people in the middle
- It's harder to evaluate software
- The misconception that AI tools are easy
- Coding agents are useful for security research now
- OpenClaw
- Journalists are good at dealing with unreliable sources
- The pelican benchmark
- And finally, some good news about parrots
- YouTube chapters

The November inflection point

4:19—The end result of these two labs throwing everything they had at making their models better at code is that in November we had what I call the inflection point where GPT 5.1 and Claude Opus 4.5 came along.

They were both incrementally better than the previous models, but in a way that crossed a threshold where previously the code would mostly work, but you had to pay very close attention to it. And suddenly we went from that to... almost all of the time it does what you told it to do, which makes all of the difference in the world.

Now you can spin up a coding agent and say, build me a Mac application that does this thing, and you'll get something back which won't just be a buggy pile of rubbish that doesn't do anything.

Software engineers as bellwethers for other information workers

5:49—I can churn out 10,000 lines of code in a day. And most of it works. Is that good? Like, how do we get from most of it works to all of it works? There are so many new questions that we’re facing, which I think makes us a bellwether for other information workers.

Code is easier than almost every other problem that you pose these agents because code is obviously right or wrong—either it works or it doesn’t work. There might be a few subtle hidden bugs, but generally you can tell if the thing actually works.

If it writes you an essay, if it prepares a lawsuit for you, it’s so much harder to derive if it’s actually done a good job, and to figure out if it got things right or wrong. But it’s happening to us as software engineers. It came for us first.

And we’re figuring out, OK, what do our careers look like? How do we work as teams when part of what we did that used to take most of the time doesn’t take most of the time anymore? What does that look like? And it’s going to be very interesting seeing how this rolls out to other information work in the future.

Lawyers are falling for this really badly. The [AI hallucination cases database](#) is up to 1,228 cases now!

Plus this bit from the cold open at [the start](#):

It used to be you’d ask ChatGPT for some code, and it would spit out some code, and you’d have to run it and test it. The coding agents take that step for you now. And an open question for me is how many other knowledge work fields are actually prone to these agent loops?

Writing code on my phone

8:19—I write so much of my code on my phone. It’s wild. I can get good work done walking the dog along the beach, which is delightful.

I mainly use the Claude iPhone app for this, both with a regular Claude chat session (which [can execute code now](#)) or using it to control [Claude Code for web](#).

Responsible vibe coding

9:55 If you're vibe coding something for yourself, where the only person who gets hurt if it has bugs is you, go wild. That's completely fine. The moment you ship your vibe coding code for other people to use, where your bugs might actually harm somebody else, that's when you need to take a step back.

See also [When is it OK to vibe code?](#)

Dark Factories and StrongDM

12:49 The reason it's called the dark factory is there's this idea in factory automation that if your factory is so automated that you don't need any people there, you can turn the lights off. Like the machines can operate in complete darkness if you don't need people on the factory floor. What does that look like for software? [...]

So there's this policy that nobody writes any code: you cannot type code into a computer. And honestly, six months ago, I thought that was crazy. And today, probably 95% of the code that I produce, I didn't type myself. That world is practical already because the latest models are good enough that you can tell them to rename that variable and refactor and add this line there... and they'll just do it—it's faster than you typing on the keyboard yourself.

The next rule though, is nobody *reads* the code. And this is the thing which StrongDM started doing last year.

I wrote a lot more about [StrongDM's dark factory explorations](#) back in February.

The bottleneck has moved to testing

21:27—It used to be, you'd come up with a spec and you hand it to your engineering team. And three weeks later, if you're lucky, they'd come back with an implementation. And now that maybe takes three hours, depending on how well the coding agents are established for that kind of thing. So now what, right? Now, where else are the bottlenecks?

Anyone who's done any product work knows that your initial ideas are always wrong. What matters is proving them, and testing them.

We can test things so much faster now because we can build workable prototypes so much quicker. So there's an interesting thing I've been doing in my own work where any feature that I want to design, I'll often prototype three different ways it could work because that takes very little time.

I've always loved prototyping things, and prototyping is even more valuable now.

22:40—A UI prototype is free now. ChatGPT and Claude will just build you a very convincing UI for anything that you describe. And that's how you should be working. I think anyone who's doing product design and isn't vibe coding little prototypes is missing out on the most powerful boost that we get in that step.

But then what do you do? Given your three options that you have instead of one option, how do you prove to yourself which one of those is the best? I don't have a confident answer to that. I expect this is where the good old fashioned usability testing comes in.

More on prototyping later on:

46:35—Throughout my entire career, my superpower has been prototyping. I've been very quick at knocking out working prototypes of things. I'm the person who can show up at a meeting and say, look, here's how it could work. And that was kind of my unique selling point. And that's gone. Anyone can do what I could do.

This stuff is exhausting

26:25—I'm finding that using coding agents well is taking every inch of my 25 years of experience as a software engineer, and it is mentally exhausting. I can fire up four agents in parallel and have them work on four different problems. And by like 11 AM, I am wiped out for the day. [...]

There's a personal skill we have to learn in finding our new limits—what's a responsible way for us not to burn out.

I've talked to a lot of people who are losing sleep because they're like, my coding agents could be doing work for me. I'm just going to stay up an extra half hour and set off a bunch of extra things... and then waking up at four in the morning. That's obviously unsustainable. [...]

There's an element of sort of gambling and addiction to how we're using some of these tools.

Interruptions cost a lot less now

45:16—People talk about how important it is not to interrupt your coders. Your coders need to have solid two to four hour blocks of uninterrupted work so they can spin up their mental model and churn out the code. That's changed completely. My programming work, I need two minutes every now and then to prompt my agent about what to do next. And then I can do the other stuff and I can go back. I'm much more interruptible than I used to be.

My ability to estimate software is broken

28:19—I've got 25 years of experience in how long it takes to build something. And that's all completely gone—it doesn't work anymore because I can look at a problem and say that this is going to take two weeks, so it's not worth it. And now it's like... maybe it's going to take 20 minutes because the reason it would have taken two weeks was all of the sort of cruffy coding things that the AI is now covering for us.

I constantly throw tasks at AI that I don't think it'll be able to do because every now and then it does it. And when it doesn't do it, you learn, right? But when it *does* do something, especially something that the previous models couldn't do, that's actually cutting edge AI research.

And a related anecdote:

36:56—A lot of my friends have been talking about how they have this backlog of side projects, right? For the last 10, 15 years, they've got projects they never quite finished. And some of them are like, well, I've done them all now. Last couple of months, I just went through and every evening I'm like, let's take that project and finish it. And they almost feel a sort of sense of loss at the end where they're like, well, okay, my backlog's gone. Now what am I going to build?

It's tough for people in the middle

29:29—So ThoughtWorks, the big IT consultancy, did an offsite about a month ago, and they got a whole bunch of engineering VPs in from different companies to talk about this stuff. And one of the interesting theories they came up with is they think this stuff is really good for experienced engineers, like it amplifies their skills. It's really good for new engineers because it solves so many of those onboarding problems. The problem is

the people in the middle. If you're mid-career, if you haven't made it to sort of super senior engineer yet, but you're not sort of new either, that's the group which is probably in the most trouble right now.

I mentioned [Cloudflare hiring 1,000 interns](#), and Shopify too.

Lenny asked for my advice for people stuck in that middle:

31:21—That's a big responsibility you're putting on me there! I think the way forward is to lean into this stuff and figure out how do I help this make me better?

A lot of people worry about skill atrophy: if the AI is doing it for you, you're not learning anything. I think if you're worried about that, you push back at it. You have to be mindful about how you're applying the technology and think, okay, I've been given this thing that can answer any question and *often* gets it right. How can I use this to amplify my own skills, to learn new things, to take on much more ambitious projects? [...]

33:05—Everything is changing so fast right now. The only universal skill is being able to roll with the changes. That's the thing that we all need.

The term that comes up most in these conversations about how you can be great with AI is *agency*. I think agents have no agency at all. I would argue that the one thing AI can never have is agency because it doesn't have human motivations.

So I'd say that's the thing is to invest in your own agency and invest in how to use this technology to get better at what you do and to do new things.

It's harder to evaluate software

The fact that it's so easy to create software with detailed documentation and robust tests means it's harder to figure out what's a credible project.

37:47 Sometimes I'll have an idea for a piece of software, Python library or whatever, and I can knock it out in like an hour and get to a point where it's got documentation and tests and all of those things, and it looks like the kind of software that previously I'd have spent several weeks on—and I can stick it up on GitHub

And yet... I don't believe in it. And the reason I don't believe in it is that I got to rush through all of those things... I think the quality is probably good, but I haven't spent enough time with it to feel confident in that quality. Most importantly, *I haven't used it yet*.

It turns out when I'm using somebody else's software, the thing I care most about is I want them to have used it for months.

I've got some very cool software that I built that I've *never used*. It was quicker to build it than to actually try and use it!

The misconception that AI tools are easy

41:31—Everyone's like, oh, it must be easy. It's just a chat bot. It's not easy. That's one of the great misconceptions in AI is that using these tools effectively is easy. It takes a lot of practice and it takes a lot of trying things that didn't work and trying things that did work.

Coding agents are useful for security research now

19:04—In the past sort of three to six months, they've started being credible as security researchers, which is sending shockwaves through the security research industry.

See Thomas Ptacek: [Vulnerability Research Is Cooked](#).

At the same time, open source projects are being bombarded with junk security reports:

20:05—There are these people who don't know what they're doing, who are asking ChatGPT to find a security hole and then reporting it to the maintainer. And the report looks good. ChatGPT can produce a very well formatted report of a vulnerability. It's a total waste of time. It's not actually verified as being a real problem.

A good example of the right way to do this is [Anthropic's collaboration with Firefox](#), where Anthropic's security team *verified* every security problem before passing them to Mozilla.

OpenClaw

Of course we had to talk about OpenClaw! Lenny had his running on a Mac Mini.

1:29:23—OpenClaw demonstrates that people want a personal digital assistant so much that they are willing to not just overlook the security side of things, but also getting the thing running is not easy. You’ve got to create API keys and tokens and install stuff. It’s not trivial to get set up and hundreds of thousands of people got it set up. [...]

The first line of code for OpenClaw was written on November the 25th. And then in the Super Bowl, there was an ad for AI.com, which was effectively a vaporware white labeled OpenClaw hosting provider. So we went from first line of code in November to Super Bowl ad in what? Three and a half months.

I continue to love Drew Breunig’s description of OpenClaw as a digital pet:

A friend of mine said that OpenClaw is basically a Tamagotchi. It’s a digital pet and you buy the Mac Mini as an aquarium.

Journalists are good at dealing with unreliable sources

In talking about my explorations of AI for data journalism through Datasette:

1:34:58—You would have thought that AI is a very bad fit for journalism where the whole idea is to find the truth. But the flip side is journalists deal with untrustworthy sources all the time. The art of journalism is you talk to a bunch of people and some of them lie to you and you figure out what’s true. So as long as the journalist treats the AI as yet another unreliable source, they’re actually better equipped to work with AI than most other professions are.

The pelican benchmark

Of course we talked about pelicans riding bicycles:

56:10—There appears to be a very strong correlation between how good their drawing of a pelican riding a bicycle is and how good they are at everything else. And nobody can explain to me why that is. [...]

People kept on asking me, what if labs cheat on the benchmark? And my answer has always been, really, all I want from life is a really good picture of a pelican riding a bicycle. And if I can trick every AI lab in the world into cheating on benchmarks to get it, then that just achieves my goal.

59:56—I think something people often miss is that this space is inherently funny. The fact that we have these incredibly expensive, power hungry, supposedly the most advanced computers of all time. And if you ask them to draw a pelican on a bicycle, it looks like a five-year-old drew it. That's really funny to me.

And finally, some good news about parrots

Lenny asked if I had anything else I wanted to leave listeners with to wrap up the show, so I went with the best piece of news in the world right now.

1:38:10—There is a rare parrot in New Zealand called the Kākāpō. There are only 250 of these parrots left in the world. They are flightless nocturnal parrots—beautiful green dumpy looking things. And the good news is they're having a fantastic breeding season in 2026,

They only breed when the Rimu trees in New Zealand have a mass fruiting season, and the Rimu trees haven't done that since 2022—so there has not been a single baby kākāpō born in four years.

This year, the Rimu trees are in fruit. The kākāpō are breeding. There have been dozens of new chicks born. It's a really, really good time. It's great news for rare New Zealand parrots and you should look them up because they're delightful.

Everyone should [watch the live stream of Rakiura on her nest with two chicks!](#)

YouTube chapters

Here's the full list of chapters Lenny's team defined for the YouTube video:

- 00:00: Introduction to Simon Willison
- 02:40: The November 2025 inflection point
- 08:01: What's possible now with AI coding
- 10:42: Vibe coding vs. agentic engineering
- 13:57: The dark-factory pattern
- 20:41: Where bottlenecks have shifted

- [23:36](#): Where human brains will continue to be valuable
- [25:32](#): Defending of software engineers
- [29:12](#): Why experienced engineers get better results
- [30:48](#): Advice for avoiding the permanent underclass
- [33:52](#): Leaning into AI to amplify your skills
- [35:12](#): Why Simon says he's working harder than ever
- [37:23](#): The market for pre-2022 human-written code
- [40:01](#): Prediction: 50% of engineers writing 95% AI code by the end of 2026
- [44:34](#): The impact of cheap code
- [48:27](#): Simon's AI stack
- [54:08](#): Using AI for research
- [55:12](#): The pelican-riding-a-bicycle benchmark
- [59:01](#): The inherent ridiculousness of AI
- [1:00:52](#): Hoarding things you know how to do
- [1:08:21](#): Red/green TDD pattern for better AI code
- [1:14:43](#): Starting projects with good templates
- [1:16:31](#): The lethal trifecta and prompt injection
- [1:21:53](#): Why 97% effectiveness is a failing grade
- [1:25:19](#): The normalization of deviance
- [1:28:32](#): OpenClaw: the security nightmare everyone is looking past
- [1:34:22](#): What's next for Simon
- [1:36:47](#): Zero-deliverable consulting

- [1:38:05](#): Good news about Kakapo parrots

How not to forget what matters

HENRIK KARLSSON · 20 JUL 2026 · [SOURCE](#)



1.

Every few months I will read a tweet, or have a conversation, that makes me feel *this is important, I must remember this*. Often, these epiphanies are accompanied by a sense that *I actually know this already*, it had just somehow slipped my mind.

And for a few days, I do remember: my life shimmers with a new intensity, and I live the truth of what I grasped. But then, inevitably, the conveyor belt of things to pay attention to keeps churning, and my mind gets filled with small problems I need to solve, or new epiphanies or random noise, like news, and the shining fades from my eyes—I regress to being the same person as ever.

The Latin word for the tendency to lose track of what matters in the cacophony of things that attract our attention is *stultia*. “Stultia,” writes Michel Foucault in “[Self-writing](#),”

is defined by mental agitation, distraction, change of opinions and wishes, and consequently weakness in the face of all the events that may occur; it is also characterized by the fact that it turns the mind toward the future, makes it interested in novel ideas, and prevents it from providing a fixed point for itself in the possession of acquired truth.

You can't just read a blog post about high agency, get filled with a sense of possibility, and become, from then on, an agentic person. As John Gray puts it in his monograph on J.S. Mill, our character is "a cluster of habitual willings." For changes to our behavior to become permanent, we must become different people.

In the same way that it is not enough to make a *resolution* that you will learn the piano, it is not enough to *realize* that when the kids act out, you shouldn't lose your temper but slow down, listen, and regulate their nervous systems with the help of yours. Imagine how good a person I would be if having insights were enough! But reacting to the frustrations of your children with calm and curiosity is a skill as much as playing the piano is—and as with the piano, the act of learning it requires rewiring your nervous system through sustained attention and practice. Realizing the value of acting in a certain way might give you a temporary motivation to do it. But in order to actually live in accordance with what you believe in long-term, you must make it a habit.

And this is much harder than making a habit out of playing the piano. When you're trying to make something like piano practice a habit, the standard advice is to chain it onto some already existing habit—to practice immediately after you brush your teeth in the morning, for example, or after you change out of your work clothes in the afternoon. Chaining the new habit to an already existing one provides a predictable trigger that helps remind you to practice. But the habits that make up our characters often do not follow a predictable schedule like this. I never know, for instance, when our children will act out (except that it will usually be when I'm least capable of handling it with grace—whenever both they and I are unusually hungry and tired). The conflicts seem to come out of nowhere, so I have to, somehow, *always* be ready to act in the proper way. I need to have the right reaction "ready at hand" (*procheiron*), as the Greek-philosopher-Roman-slave Epictetus put it. If Johanna and I talked about how we want to deal with the kids' conflicts the night before, I will nearly always handle the situation well. The problem is to keep it top of mind.

2.

During the first two centuries of the Roman Empire, there spread a practice known as *hypomnēmata*, a type of notetaking system, used as a tool for meditation,^[1] in which the writer would store quotes from books they had read. Each day, often in the morning, the notetaker would open their notebook and look for a passage relevant to something they were struggling with, and then they would meditate on that—unpacking it, making the idea top of mind, ensuring it was alive in them. If they needed courage, for instance, they could meditate on an anecdote that made it real for them what it meant to act bravely. The idea was that over time, the insights they gathered by reading would be transformed into character, something deeply ingrained in their way of thinking and seeing and acting.

This was, as I understand it, an exercise designed to combat the problem I outlined above. Meditating on what matters is a simple habit, which you can chain onto your morning routine, but it reinforces the habits you can't plan, the habits that make up your character. It was, in the words of the French classicist Pierre Hadot, a spiritual exercise—an *exercise* because it required work and discipline, *spiritual* because it engaged the whole person, not just their intellect, but their emotions and their moral character. It was an attempt to treat the formation of character as a skilled practice, as something you can deliberately train and improve through targeted exercises.

The most famous example of this practice is the meditations Marcus Aurelius wrote in his tent as plague swept through the camps during the military campaigns along the Danube River, but it seems to have been a fairly widespread practice among the “cultivated class.” A suggestion repeated in several popular manuals for living was that you should collect every snippet of thought that deeply inspires you to live in a more ethically true way and then, in the pre-dawn hour, look through your hypomnēmata to find passages relevant to your current situation—insightful quotes, examples, actions you had witnessed, notes from conversations you'd had, and so on^[2]—and meditate, in writing, on those that help you orient toward your current challenges, until you feel inspired to act in the proper way.^[3]

What could be an example? Today, June 14, I woke up with a headache after having slept with my neck at an odd angle and so didn't feel like working. Having procrastinated for an hour or so, while Johanna tried to get me to start the day, I recalled principle 23 from [a list Nabeel S. Qureshi has compiled](#) for himself:

Doing things is energizing, wasting time is depressing. You don't need that much 'rest'.

Walking around the farm with a cup of coffee,^[4] reminding myself of the truth of this observation, I gained a sliver of motivation, enough to throw myself into rewriting this essay—and now my headache has lifted, and I feel excited. For more examples of things one might meditate on, I suggest looking at Nabeel's list. I also like to meditate on stories that make real to me some ethos I aspire to, such as the story of how Werner Herzog, dead broke while scouting for locations for *Nosferatu* in Brittany, happened upon a field of menhirs and decided to abandon his work to stay at the field for as long as necessary to solve the mystery of how the giant stones had been erected—a story that makes it visceral to me what it means to “take your curiosity seriously.”

The hypomnēma was a mechanism for centering your mind on what matters, and for gradually refining your understanding of what matters. It was, to use a word from Plutarch, a tool for *ethopoiesis* (*ethos* meaning “character” and *poiesis* “making”), a tool for turning truth into character. By meditating daily on sentences that made it real to you how you wanted to live, you

would remember to do the right thing—you would remember to practice it during the day (simply thinking about it is not going to change you). And gradually, you would become that sort of person. You wouldn't even need to remind yourself. Your principles would have become your character. You would have developed expertise in the skill of holding yourself in a better way.

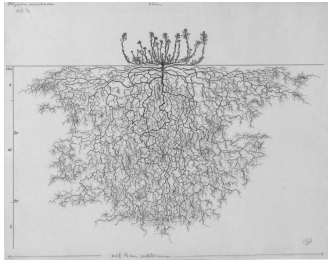
3.

I first came across the idea of using hypomnēmata in this way eight years ago when a friend sent me a copy of Foucault's "Self-writing." In the essay, Foucault talks about hypomnēmata and other modes of reading and writing used to fashion a self during the Hellenistic and Imperial eras. I didn't pick up the practice in its full form. But rereading the essay now, I realize how much it has influenced how I write: my essays are (often) meditations I do in order to deepen core ideas I want to live by, and to strengthen parts of myself I want strengthened.^[5] (Writing treatises and letters was a common way of internalizing the content of the hypomnēmata.) And this practice has been transformative for me. I have often noticed that my experience of reality improves if I write and think about something.

But it strikes me now that the practice Foucault wrote about was probably more transformative than what I've ended up doing. Essay writing is incredibly time-consuming, and a lot of that time is spent on things that aren't self-transforming: I spend less time reshaping my mind than I spend solving literary-technical problems that help me write more functional and beautiful essays, for the joy of the craft and for the benefit of readers. Another limitation of my practice is that when an essay is done, I move on. The ideas—though they have been much deepened and more firmly lodged in my mind—fall out of attention and start to fade.

There is an element of self-deception involved here. I like to write essays, so it is comforting to think of it as a powerful practice, something that helps me live more fully and grow as a person. But if I look at it soberly, it is clear to me that essay writing is not a practice that is ideal for the purpose of ethopoiesis.^[6] It is common to think that what we do achieves what we want it to achieve, even if there is no evidence for it. There are many practices that promise to transform and improve us—therapy, meditation, psychedelics—, but that branding doesn't mean that they actually do much for us: it is common to see people use these techniques for years without any obvious progress on their problems. If you want to achieve a particular outcome, it is important to start from that goal and evaluate which practices actually help you.

The most important ideas we need to return to weekly, even daily. Essay-writing, then, is not a functional substitute for having a practice that keeps the important truths top of mind, day after day. But it did help me reach that conclusion.



How to think in writing

Credits

This essay is based on a fragment from a longer essay on spiritual exercises that I wrote. That essay didn't work, and Johanna suggested using a few of the paragraphs for a new essay. She also suggested the main argument and general outline of the body of this essay, reorganized the second draft to shift the emphasis to what we came to see as the main idea, and corrected several faulty lines of argument I made.

*The ideas were influenced by reading Pierre Hadot's *Philosophy as a Way of Life* and Michel Foucault's *The Hermeneutics of the Subject*, *The Care of the Self*, and "[Self-Writing](#)." I was influenced by several conversations with Michael Nielsen and his [notes on Tanya Luhrmann's How God Becomes Real](#), as well as a conversation with Alexander Obenauer about his practice. Claude Fable 5 (RIP) corrected four factual errors.*

1 [find in text]

The practice of keeping hypomnēmata had already existed for more than 400 years at this point, but the practices around it shifted significantly around this era, as I understand it, from a more conventional type of notetaking into the more specialized type of meditation described in this essay.

2 [find in text]

How do you actually keep these notes? A lot of people have insights and write them into Apple Notes, but then they never look at them again, and it becomes a mess after a while, so when they need the idea, it is nowhere to be found. I'm not sure exactly how the Romans managed to find their way back to the right wax tablet or papyrus. I guess the easiest thing is to just keep a single document for the most important ideas and stories, with short memorable concept handles (länks) that help you remember each. If you can't be bothered making your notes structured, though, LLMs are powerful enough now that they can trawl through your messy notes and find the

relevant quotes. Just give Claude Code or Codex access to your notes and ask it to find segments relevant to whatever you are thinking about. This will not be as powerful as doing the work yourself, but it is better than just forgetting.

3 [find in text]

I read somewhere that when meditating on their hypomnēmata, the Romans and Hellenistic Greek would also set intentions for themselves, principles they want to live by that day. I can't find the source for this claim now, but it resonated with something Johanna often tells me: it is much easier to hold the line if you've articulated your position to yourself beforehand. If, for example, I'm on a call with someone who wants to propose a project, I will often get caught up in their excitement, or feel uncomfortable setting the right boundaries—I often don't even know what the right boundaries are, because there is too much to consider for me to figure out my position in real time—and so I will say yes to things I shouldn't. But if I have thought things through beforehand, and have articulated to myself what I'm trying to achieve, and what my principles are, I can more easily see if this is in line with what I want, and I can clearly articulate my position. I wish I did this more often. Or at least remembered the principle to always ask for time to think things through on my own before committing.

4 [find in text]

Seneca and Epictetus would have insisted on making meditating in writing. The argument is that writing engages you more deeply, transforming what you have read “into tissue and blood” (in vivres et in sanguinem). Using an oft-repeated metaphor, Seneca writes, “We should imitate bees, who wander and pluck from flowers suited for making honey, then arrange and distribute through the combs what they have brought in... We must digest what we have read; otherwise it passes into the memory, not into the mind.” Food, he says, is a burden so long as it stays what it was; only when it changes into blood and tissue does it become strength. It might feel like I understand a topic without writing about it, but if I earnestly try to unpack my thinking in writing, my ignorance is revealed, and my thinking transformed.

5 [find in text]

I'm not sure how much of the shift in my writing came directly from reading Foucault: it has been something that I've iterated myself toward. I've gradually written more with the intention of shaping my character and my attention, and this has been driven by the experience of deepened presence this has afforded me. But I'm pretty sure the ideas from that essay provided a template that helped me make sense and direct the evolution of my writing. I know I have often returned to the essay, and have used Plutarch's term ethopoetical writing to make sense to what I'm trying to do.

What it can do well, though, is to push deep into an idea and to restructure your thoughts about something. It is like an intervention where you want to shake things up properly, a three week session where you think and think and think about the same thing all the time. It is, I think, a useful complement to a daily practice.

2.1.241

CHANGELOG · 23 AUG 2026 · [SOURCE](#)

- Bug fixes and reliability improvements
-

2.1.240

CHANGELOG · 22 AUG 2026 · [SOURCE](#)

- Bug fixes and reliability improvements